

Retail Data Analytics Chat System - Homework Project

Project Overview

Build an AI-powered retail analytics system that allows users to query transaction data through a conversational interface. The system should integrate the Kaggle Retail Transaction Dataset with an LLM to answer both specific (customer/product-level) and aggregated business questions.

Dataset: <https://www.kaggle.com/datasets/fahadrehman07/retail-transaction-dataset/data>

Note: Feel free to use AI coding tools (such as Cursor, etc.) for this assignment.

Core Requirements

1. Data Layer

- Download and load the Kaggle Retail Transaction Dataset
- Store data in a database or file-based system (your choice)
- Implement data access layer (a RPC or REST API) to query:
 - Customer-specific information (by CustomerID)
 - Product-specific information (by ProductID)
 - Aggregated metrics for business intelligence

2. LLM Integration

- Integrate any LLM (ChatGPT API, local models like Ollama, etc.)
- Implement a system that:
 - Parses user intent (identify if asking about customer, product, or business metrics)
 - Extracts relevant IDs or parameters from natural language queries
 - Retrieves appropriate data from the dataset
 - Generates natural language responses based on data

3. Chat Interface

- Create a chat-based UI for querying the system
- Minimal requirement: these types of queries:
 - A. Customer-Specific Queries (input: customer_id)
 - Examples:
 - "What is the purchase history for customer C67890?"
 - "How much has customer C11111 spent in total?"

B. Product-Specific Queries (input: product_id)

- Examples:
 - "What's the average discount for product P1234?"
 - "Which stores sell the product P9999?"

4. Query Processing Logic

- Implement intent classification to route queries appropriately
- Extract entities (customer IDs, product IDs) from natural language
- Execute appropriate SQL/data queries based on user intent
- Handle edge cases (invalid IDs, ambiguous queries, no data found)

5. **Stretch goal**: Enhance query flexibility to support a broader range of scenarios and unlock more use cases.

Technical Stack Suggestions

- Frontend: Vue.js, React, or Streamlit
- Backend: Python (FastAPI/Flask), Node.js, or Go
- Database: SQLite, PostgreSQL, MySQL, or Pandas DataFrames
- LLM: OpenAI API, Anthropic Claude, Ollama (local), or any other LLM

Note: These are suggestions only. Use any technology stack you're comfortable with.

Testing Requirements

- End-to-end testing for core query flows:
 - Customer query → data retrieval → response generation
 - Product query → data retrieval → response generation
 - Business question → aggregation → response generation
- Unit tests for core logic, no need high coverage

Deliverables

Code Submission

- Complete code checked into GitHub repository
- **Share repository access with: gensparkreviewer@gmail.com**
- Include README.md with:
 - Setup instructions for local run
 - Dataset download/setup instructions
 - Environment variables needed (API keys, etc.)
 - Example queries to test the system
- **Screenshots or screen recording of the system in action**

Example Queries to Support

Your README should include examples like:

- "What has customer C12345 purchased?"
- "How much has customer C11111 spent in total?"
- "Which stores sell the product P9999?"

Extra Bonus Ideas

- Support follow-up questions with context awareness
- Add comparison queries ("Compare customer A vs customer B")
- Add data visualizations (charts/graphs) for aggregated queries

Review Process

Review Meeting Preparation

- Prepare bullet points explaining:
 - How you integrated the LLM with the data layer
 - Your approach to query parsing and intent classification
 - Data modeling and storage decisions
- Be ready to demonstrate:
 - Live querying with different question types
 - How the system handles edge cases

No comprehensive documentation required - focus on key technical decisions.

Evaluation Criteria

1. Functionality (40%)
 - All three query types working correctly
 - Accurate data retrieval and presentation
 - LLM integration producing meaningful responses
2. Data Integration (20%)
 - Proper data loading and storage
 - Correct querying for customer/product lookups
3. LLM Integration (20%)
 - Effective use of LLM for query understanding
 - Proper prompt engineering for data-driven responses
4. Testing (10%)
 - Critical paths
 - Edge case handling
5. Code Quality & Technical Decisions (10%)
 - Clean, maintainable code
 - Ability to explain architecture and implementation choices

Good luck! We're excited to see your approach to building a data-driven AI system.